PYTHON THEORY

Python is a high-level, general-purpose and a very popular programming language. Python programming language (latest Python 3) is being used in web development, Machine Learning applications, along with all cutting edge technology in Software Industry. Python Programming Language is very well suited for Beginners, also for experienced programmers with other programming languages like C++ and Java.

- **SOME FACTS ABOUT PYTHON PROGRAMMING LANGUAGE**

- Python is currently the most widely used multi-purpose, high-level programming language.
- Python allows programming in Object-Oriented and Procedural paradigms.
- Python programs generally are smaller than other programming languages like Java. Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.
- Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber… etc.

ADVANTAGES OF PYTHON

1. Presence of third-party modules
2. Extensive support libraries(NumPy for numerical calculations, Pandas for data analytics, etc.)
3. Open source and large active community base
4. Versatile, Easy to read, learn and write
5. User-friendly data structures
6. High-level language
7. Dynamically typed language(No need to mention data type based on the value assigned, it takes data type)

Applications:

1. GUI-based desktop applications
2. Graphic design, image processing applications, Games, and Scientific/ computational Applications
3. Web frameworks and applications
4. Enterprise and Business applications
5. Operating Systems
6. Education
7. Database Access
8. Language Development
9. Prototyping

10. Software Development

PYTHON BASICS:

Python was developed by Guido van Rossum in the early 1990s and its latest version is 3.10.4, we can simply call it Python3. Python 3.0 was released in 2008. and is interpreted language i.e it's not compiled and the interpreter will check the code line by line.

VARIABLES AND DATA STRUCTURES:

In other programming languages like C, C++, and Java, you will need to declare the type of variables but in Python you don't need to do that. Just type in the variable and when values will be given to it, then it will automatically know whether the value given would be an int, float, or char or even a String.

FOR EXAMPLE:

```
Python program to declare variables
myNumber = 3
print(myNumber)

myNumber2 = 4.5
print(myNumber2)

myNumber ="helloworld"
print(myNumber)
```

- Python have 4 types of built in Data Structures namely List, Dictionary, Tuple and Set.

List is the most basic Data Structure in python. List is a mutable data structure i.e items can be added to list later after the list creation. It's like you are going to shop at the local market and made a list of some items and later on you can add more and more items to the list.
append() function is used to add data to the list.

```
# Python program to illustrate a list

# creates a empty list
nums = []

# appending data in list
```

```
nums.append(21)
nums.append(40.5)
nums.append("String")

print(nums)
```

Comments:

```
# is used for single line comment in Python
""" this is a comment """ is used for multi line comments
```

NUM1 = 93:

Set the value of a variable, if there is not a variable already created, it will create one. A variable is a container for a value (in this case the variable will be called "num1" and store the value 93). The value stored in the variable can change while the program is running. The variable can be called whatever you want (except Python dedicated words such as print, save, etc.) and it must start with a letter rather than a number or symbol and have no spaces.

ANSWER = NUM1 + NUM2 :Adds together num1 and num2 and stores the answer in a variable called answer.

ANSWER = NUM1 - NUM2 :Subtracts num2 from num1 and stores the answer in a variable called answer.

ANSWER = NUM1 * NUM2 :Multiplies num1 by num2 and stores the answer in a variable called answer.

ANSWER = NUM1 / NUM2 :Divides num1 by num2 and stores the answer in a variable called answer.

ANSWER = NUM1 // NUM2: A whole number division (i.e. 9//4 = 2) and stores the answer in a variable called answer.

1. **PRINT ("THIS IS A MESSAGE"):**

Displays the message in the brackets. As the value we want displayed is a text value it has the speech marks, which will not be displayed in the output. If you wanted to display a numerical value or the contents of a variable, the speech marks are not needed.

2. **PRINT ("FIRST LINE\NSECOND LINE"):**
   "\n" is used as a line break.
3. **PRINT ("THE ANSWER IS", ANSWER):**
   Displays the text "The answer is" and the value of the variable answer.

4. **TEXTVALUE = INPUT("ENTER A TEXT VALUE: ") :**

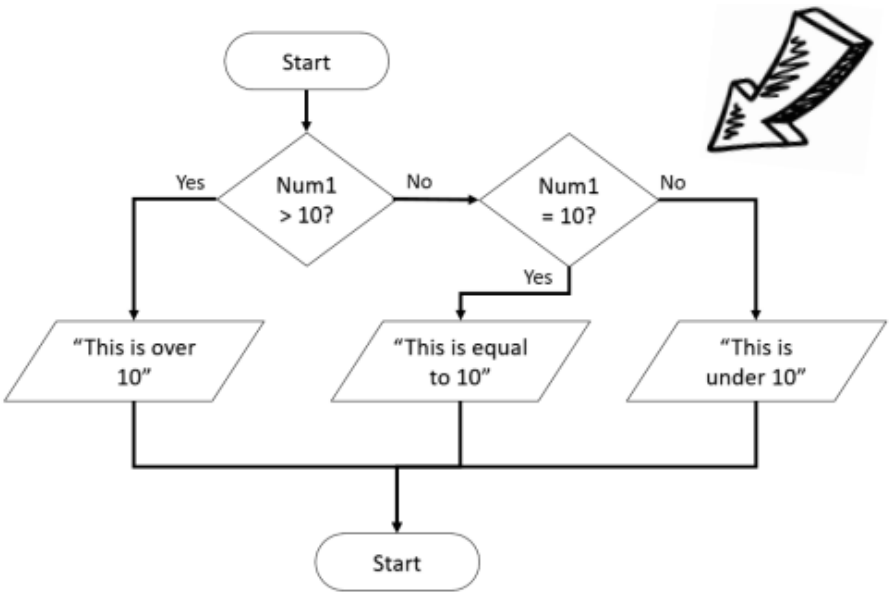
Displays the question “Enter a text value: ” and stores the value the user enters in a variable called textValue. The space after the colon allows a space to be added before the user enters their answer, otherwise they appear squashed unattractively together.

5. **NUMVALUE = INT(INPUT(“ENTER A NUMBER: ”)) :**

Displays the question “Enter a number: ” and stores the value as an integer (a whole number) in a variable called numValue Integers can be used in calculations but variables stored as text cannot.

IF STATEMENTS:

If statements allow your program to make a decision and change the route that is taken through the program.




```
if num1 > 10:
    print("This is over 10")
elif num1 == 10:
    print("This is equal to 10")
else:
    print("This is under 10")
```

Indenting Lines of Code:

Indenting is very important in Python as it shows the lines that are dependent on others, as shown in the example on the previous page. In order to indent text you can use your [Tab] key or you can press your [space key] five times. The [backspace] key will remove indents. The first line of the if statement

tests a condition, and if that condition is met (i.e. the first condition is true) then the lines of code directly below it are run. If it is not met (i.e. the first condition is false) it will test the second condition, if there is one, and so on. Below are examples of the different comparison and logical operators you can use in the condition line of your if statement.

OPERATORS IN PYTHON:

Comparison Operators

Operator	Description
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Logical Operators

Operator	Description
and	Both conditions must be met
or	Either condition must be met

- **IF-ELSE:**

```
if num > 10:
 print("This is over 10")
else:
 print("This is not over 10")
```

If num1 is over 10, it will display the message "This is over 10", otherwise it will display the message "This is under 10".

- **NESTED IF ELSE:**

```
if num >= 10:
 if num <= 20:
  print("This is between 10 and 20")
 else:
  print("This is over 20")
else:
 print("This is under 10")
```

If num1 is 10 or more then it will test another if statement to see if num1 is less than or equal to 20. If it is, it will display the message "This is between 10 and 20". If num1 is not less than or equal to 20 then it will display the message "This is over 20". If num1 is not over 10, it will display the message "This is under 10".

- **IF-ELSE-LADDER:**

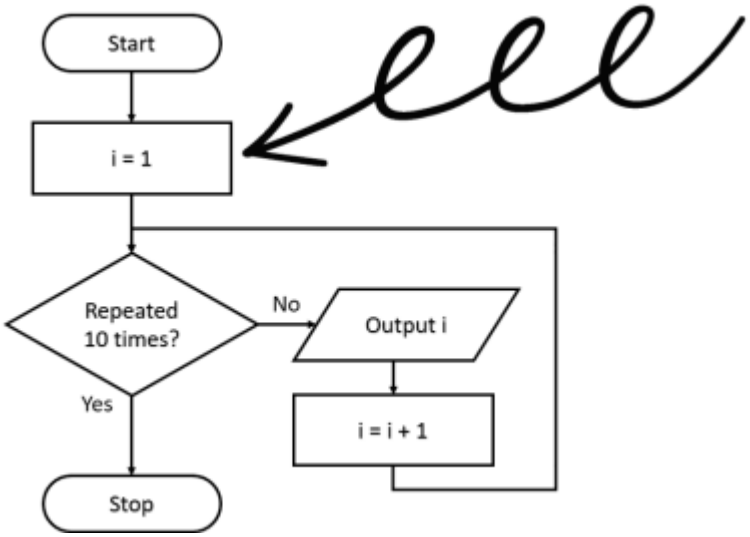
```
if num > 10:
 print("This is over 10")
elif num == 10:
 print("This is equal to 10")
else:
 print("This is under 10")
```

If num1 is over 10, it will display the message "This is over 10", otherwise it will check the next condition. If num1 is equal to 10, it will display the message "This is equal to 10". Otherwise, if neither of the first two conditions have been met, it will display the message "This is under 10".

LOOPS:

In computer programming, a loop is **a sequence of instruction s that is continually repeated until a certain condition is reached**. Typically, a certain process is done, such as getting an item of data and changing it, and then some condition is checked such as whether a counter has reached a prescribed number.

- FOR LOOP:
  A for loop allows Python to keep repeating code a set number of times. It is sometimes known as a counting loop because you know the number of times the loop will run before it starts.



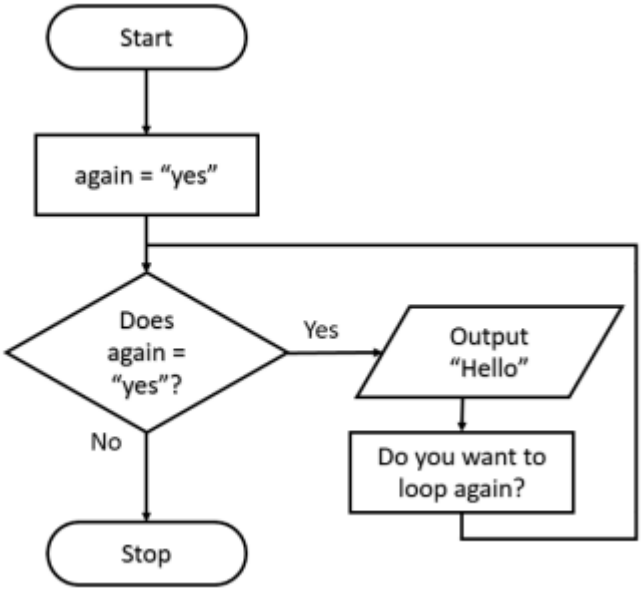

```
for i in range(1,10):
 print(i)
```

This loop uses a variable called "i" to keep track of the number of times the loop has been repeated. It will start i at 1 (as that is the starting value in the range function) and repeat the loop, adding 1 to i each time and displaying the value of i until it reaches 10 (as dictated by the range function), where it will stop. Therefore, it will not repeat the loop a tenth time and will only have the following output:

```
1, 2, 3, 4, 5, 6, 7, 8, 9
```

WHILE LOOP:

- A while loop allows code to be repeated an unknown number of times as long as a condition is being met. This may be 100 times, just the once or even never. In a while loop the condition is checked before the code is run, which means it could skip the loop altogether if the condition is not being met to start with. It is important, therefore, to make sure the correct conditions are in place to run the loop before it starts.





```
total = 0
while total < 100:
 num = int(input("Enter a number: "))
 total = total + num
print("The total is", total)
```

The above program will create a variable called total and store the value as 0. It will ask the user to enter a number and will add it to the total. It will keep repeating this as long as the total is still below 100. When the total equals 100 or more, it will stop running the loop and display the total.



STRINGS AND NUMBERS AS VARIABLES:

If you define a variable as a string, even if it only contains numbers, you cannot later use that string as part of a calculation. If you want to use a variable that has been defined as a string in a calculation, you will have to convert the string to a number before it can be used.

IMPORTANT:

`len(word)`
Finds the length of the variable called word.

`print(word.capitalize())`
Displays the variable so only the first word has a capital letter at the beginning and everything else is in lower case.

`name = firstname+surname`
Joins the first name and surname together without a space between them, known as concatenation

`word.upper()`
Changes the string into upper case.

`word.lower()`
Changes the string into lower case.

`word.title()`
Changes a phrase so that every word has a capital letter at the beginning with the rest of the letters in the word in lower case (i.e. Title Case).

```
text = " This is some text. "
print(text.strip(" "))
```
Removes extra characters (in this case spaces) from the start and end of a string.